E0 — Entornos virtuales y control de versiones (Git/GitHub)

Explicación de la práctica y de los conceptos teóricos para repasar de cara al examen.

1. Qué se ha hecho en la práctica

Objetivo: dejar preparada la base **reproducible** sobre la que se construye toda la asignatura: un entorno virtual de Python aislado y un repositorio personal en GitHub con historial limpio.

Paso	Qué se hizo
1. Entorno virtual	Crear `.venv`, activarlo, instalar dependencias y verificarlo con `check_entorno.py --strict` → `ENTORNO OK`
2. Repo personal	Crear `dsia-26-27-VizosoMarchena-Gonzalo` en GitHub y clonarlo **al lado** del repo del curso (no dentro)
3. `.gitignore`	Copiar la plantilla del curso para que no se suban `.venv/`, `.env`, `__pycache__/`...
4. Documentación	`README.md` y `sesion01.md` con la autoevaluación de 8 preguntas
5. Flujo Git	Rama `practica/sesion-01` → `add` → `commit` → `push` → Pull Request a `main`

Checklist del enunciado y cómo se cumple:

#	Paso del enunciado	Comando o evidencia
1	Crear y activar el venv e instalar dependencias	`python -m venv .venv` → `source .venv/Scripts/activate` → `pip install -r requirements.txt`
2	Verificar el entorno	`python 1_programacion_avanzada_python/ejemplos/check_entorno.py --strict` → `ENTORNO OK`
3	Repo `dsia-26-27-apellido-nombre` + `.gitignore`	`dsia-26-27-VizosoMarchena-Gonzalo` + plantilla `ejemplos/gitignore_dsia.txt` copiada como `.gitignore`
4	Rama `practica/sesion-01` con `sesion01.md` y `README.md`	`git switch -c practica/sesion-01` + commit de los dos ficheros
5	Pull Request hacia `main`	PR #1 "Sesión 01: entorno y Git" en GitHub

"Hecho cuando...": URL del repo y PR visibles, venv en verde con `--strict` y notas conceptuales en `sesion01.md`.

Qué hace `check_entorno.py`: comprueba la versión de Python y que se está ejecutando **dentro** de un `.venv`. Sin `--strict` solo muestra un aviso si no detecta el entorno virtual; **con `--strict` falla**, así que obliga a tener el venv activado. Ejercicio de la teoría: ejecutarlo

dentro del venv (OK), hacer `deactivate`, ejecutarlo de nuevo (aviso o fallo) y fijarse en que cambia `sys.executable`.

El paso 1 se hace en el repo del curso (que trae `requirements.txt` y `check_entorno.py`); los pasos 3 a 5 en el **repo personal**.

Organización de carpetas:

```
Desarrollo de Soluciones de IA/
  dsia-26-27/                          # repo del profesor
  dsia-26-27-VizosoMarchena-Gonzalo/   # repo personal
```

Los dos repos van **al mismo nivel**, nunca uno dentro de otro, porque Git se confundiría con un repositorio dentro de otro. En el repo del profesor no se puede hacer `push` (da error `403 Permission denied`).

---

2. Conceptos teóricos: entornos virtuales

2.1 Reproducibilidad

En DSIA no se entrega "un script que funciona en mi portátil", sino una solución que otra persona pueda **clonar, instalar, ejecutar y revisar**. Para ello hacen falta:

> **mismo código + mismas dependencias + historial**

Regla de oro: lo que es secreto (`.env`) o se puede regenerar (`.venv`) **no** va a Git. Lo que define el proyecto (código, tests, README, `requirements.txt`, `.env.example`) **sí**.

2.2 Qué es un entorno virtual (`venv`)

Es una carpeta (normalmente `.venv/`) con:

- un intérprete de Python propio del proyecto,
- su propio `pip`,
- su propio `site-packages` (la carpeta donde se instalan las librerías),
- scripts de activación.

Qué problema resuelve:

Sin venv	Con venv
El proyecto A necesita `pandas 2.0` y el B necesita `2.2`: conflicto	Cada proyecto tiene sus propias versiones
"En clase funciona, en casa no"	Mismas dependencias gracias a `requirements.txt`
Actualizar una librería rompe otro proyecto	El daño queda aislado

Analogía: el venv es una caja de herramientas **para cada obra**, y `requirements.txt` es la lista de la compra para montar la misma caja en otro sitio.

2.3 Ciclo de vida del venv

```
python -m venv .venv                    # crear
source .venv/Scripts/activate           # activar (Git Bash en Windows)
.venv\Scripts\activate                  # activar (cmd / PowerShell)
source .venv/bin/activate               # activar (macOS / Linux)
python -m pip install -r requirements.txt   # instalar dependencias
deactivate                              # salir (no borra nada)
```

- Tras activarlo, el prompt muestra `(.venv)`.
- Si el entorno se corrompe, se borra la carpeta `.venv` y se vuelve a crear.
- `.venv/` **nunca se sube a GitHub:** ocupa cientos de MB y se puede regenerar.

Cómo comprobar que se está dentro del venv:

```
python -c "import sys; print(sys.executable)"   # la ruta debe contener .venv
where python                                    # Windows
echo $VIRTUAL_ENV                               # Git Bash / macOS / Linux
```

2.4 `python -m pip` frente a `pip`

`pip` instala los paquetes **en el Python que lo ejecuta**. Si hay varios Python instalados, `pip` y `python` pueden no corresponderse. `python -m pip install ...` garantiza que el paquete se instala en el mismo Python con el que se va a ejecutar el código.

2.5 `requirements.txt`

Estilo	Ejemplo	Ventaja / inconveniente
Rangos mínimos	`pandas>=2.2`	Flexible y legible; dos personas pueden acabar con versiones algo distintas
Versión exacta (*pin*)	`pandas==2.2.3`	Máxima reproducibilidad; más trabajo al actualizar
Lock generado	`pip freeze > requirements-lock.txt`	Foto exacta del entorno; incluye también las dependencias indirectas (ruidoso)

Buena práctica: listar solo las **dependencias directas** que importa el código.

2.6 El IDE y el intérprete

Aunque la terminal use `.venv`, VS Code puede usar otro Python. Se corrige con `Ctrl+Shift+P` → **Python: Select Interpreter** → `.venv\Scripts\python.exe`. Si la terminal importa `pandas` y el IDE no, el 99 % de las veces es que el intérprete seleccionado es otro.

2.7 Secretos y `.env`

```
.env            → claves reales (NO va a Git)
.env.example    → nombres de las variables, sin valores (SÍ va a Git)
python-dotenv   → carga el .env al ejecutar en local
```

Nunca escribir `API_KEY = "sk-..."` en el código.

Si se filtra un secreto: lo primero es **revocar la clave** en el proveedor. Borrarla con un commit nuevo no basta, porque sigue en el historial.

2.8 Anti-patrones de entornos

1. `sudo pip install` en el Python del sistema.
2. Subir `.venv/` a GitHub.
3. No saber qué Python usa el IDE.
4. Instalar paquetes sin activar el entorno.
5. Mezclar conda, pip global y venv sin criterio.

---

3. Conceptos teóricos: Git y GitHub

3.1 Git frente a GitHub

- **Git:** sistema de control de versiones **distribuido**. Cada clon tiene el historial completo en la carpeta `.git/` y se puede trabajar sin conexión.
- **GitHub:** plataforma que aloja repositorios Git y añade Pull Requests, issues y Actions (CI). **Git ≠ GitHub.**

3.2 Las tres zonas y el remoto

```
working tree  --git add-->  staging  --git commit-->  commits locales  --git push-->  remoto (origin)
```

Zona	Contenido
Working tree	Los ficheros tal como están en el disco
Staging (*index*)	Lo que **entrará** en el próximo commit
Commits locales	El historial guardado en `.git/`
Remoto (`origin`)	La copia en GitHub

```
git status          # estado de las tres zonas
git diff            # cambios que aún no están en staging
git diff --cached   # cambios en staging (lo que entrará en el commit)
git log --oneline   # historial resumido
```

3.3 Glosario

Término	Definición
Commit	Foto del proyecto + mensaje + autor + fecha
Hash (SHA)	Identificador único de un commit
Branch (rama)	Línea de trabajo paralela; puntero móvil a una serie de commits
`HEAD`	"Dónde estoy ahora"
`origin`	Nombre por defecto del remoto principal
Clone	Copiar un repo remoto al ordenador (se hace una vez)
Fetch	Traer commits del remoto **sin** fusionarlos
Pull	Fetch + integrar los cambios en la rama actual
Push	Subir los commits locales al remoto
Merge	Combinar dos historias
Conflicto	Dos cambios incompatibles en las mismas líneas
Fork	Copia de un repo en otra cuenta de GitHub
Pull Request (PR)	Propuesta de integrar una rama en otra, con revisión

Fork, clone y repo propio: *clone* descarga un repo a tu ordenador; *fork* crea una copia en tu cuenta de GitHub; en esta asignatura el repo personal se **crea desde cero** (no es un fork) y se clona.

3.4 Buenos commits

- **Atómicos:** un solo propósito por commit.
- **Mensaje imperativo y concreto:**

Bien	Mal
`Add sales validator tests`	`update`
`Fix README install steps`	`cambios`
`Ignore venv and env files`	`fix final`

3.5 Ramas y Pull Requests

```
git switch -c practica/sesion-01       # crear una rama y cambiarse a ella
git add README.md sesion01.md .gitignore
git commit -m "Add session 01 notes and gitignore"
git push -u origin practica/sesion-01  # -u vincula la rama local con la remota
```

Después, en GitHub: **Compare & pull request** → base `main` ← `practica/sesion-01` → descripción → **Merge**. Y en local:

```
git switch main
git pull
```

Nombres de rama: `practica/...`, `feature/...`, `fix/...`, `docs/...`.

¿Por qué usar ramas aunque se trabaje solo? `main` siempre queda estable, cada trabajo queda aislado y la PR permite revisar los cambios antes de integrarlos.

3.6 `.gitignore`

```
.venv/
.env
.env.*
!.env.example        # "!" = excepción: este sí se sube
__pycache__/
*.py[cod]
.pytest_cache/
.ipynb_checkpoints/
.vscode/
```

Si ya se subió algo por error:

```
git rm -r --cached .venv     # lo deja de seguir SIN borrarlo del disco
git commit -m "Stop tracking local environment"
```

3.7 `git add` con intención

`git add .` añade **todo**, y es fácil colar `.venv`, datos grandes o secretos. Es más seguro añadir ficheros concretos (`git add fichero1 fichero2`) y revisar siempre `git status` antes del commit. Para sacar un fichero del staging: `git restore --staged fichero`.

3.8 Deshacer cambios

Quiero...	Comando	Riesgo
Descartar cambios no guardados de un fichero	`git restore fichero`	Se pierden esas ediciones
Sacar un fichero del staging	`git restore --staged fichero`	Bajo
Corregir el último commit (antes del push)	`git commit --amend`	Medio
Deshacer un commit ya subido	`git revert HASH` (crea un commit nuevo que lo deshace)	Bajo, el más seguro
Mover HEAD hacia atrás	`git reset`	Alto si ya hay push

3.9 Problemas frecuentes

Síntoma	Solución
`rejected (fetch first)`	`git pull` y después `git push`
`403 Permission denied`	Estás en un repo sin permiso de escritura (por ejemplo, el del profesor)
`Your branch is ahead of origin/main`	Falta hacer `git push`
Aparece `.venv` en el commit	Revisar `.gitignore` + `git rm -r --cached .venv`
`nothing to commit, working tree clean`	No hay cambios en esa carpeta; comprueba en qué repo estás

3.10 Configuración inicial (una vez por ordenador)

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu@email.es"
git config --global init.defaultBranch main
```

---

4. Uso de IA

Cuando un asistente (Claude, ChatGPT, Cursor...) genera partes relevantes del trabajo, hay que:

1. Revisar el código y no copiarlo a ciegas.
2. Comprobar que no se inventa funciones o APIs.
3. **Documentarlo** (en la asignatura, en un fichero `AI_USAGE.md`): qué herramienta se usó y qué se aceptó o rechazó.
4. No pegar nunca secretos en los prompts.

Usar IA sin citarlo se considera **plagio** según el reglamento de la universidad.

---

5. Preguntas típicas de examen (E0)

1. **¿Qué diferencia hay entre el Python global y un `.venv`?** El global es compartido por todo el sistema; el `.venv` es un Python aislado con sus propias librerías, solo para ese proyecto.
2. **¿Para qué sirve `python -m pip` frente a `pip`?** Para asegurar que el paquete se instala en el mismo Python con el que se ejecuta el código.
3. **Explica working tree, staging y commit.** El working tree son los ficheros en el disco; el staging es lo que se ha seleccionado con `git add` para el próximo commit; el commit es la foto guardada en el historial.

4. **¿Clone y pull son lo mismo?** No: `clone` descarga el repo completo por primera vez; `pull` trae e integra los cambios nuevos en un repo que ya tienes.
5. **Cuatro cosas que no se suben a GitHub.** `.venv/` (se puede regenerar y es enorme), `.env` y claves API (son secretos), datos personales o confidenciales y ficheros enormes innecesarios.
6. **Reescribe bien `update`, `fix final` y `cambios varios`.** Por ejemplo: `Add pandas exercise E1`, `Fix CSV path in loader`, `Update README with test commands`.
7. **¿Qué haces si el IDE no importa `pandas` pero la terminal sí?** Seleccionar el intérprete del `.venv` en el IDE (*Python: Select Interpreter*).
8. **¿Qué haces si subiste el `.env` por error?** Revocar la clave inmediatamente, dejar de seguir el fichero con `git rm --cached .env`, añadirlo al `.gitignore` y hacer commit. Borrarlo sin más no basta, porque sigue en el historial.
9. **¿Qué diferencia hay entre fetch y pull?** `fetch` solo descarga los commits remotos; `pull` los descarga y además los integra en tu rama.
10. **¿Por qué `revert` es más seguro que `reset` cuando ya se ha hecho push?** Porque `revert` crea un commit nuevo que deshace el cambio sin reescribir el historial compartido.
11. **¿Qué es una Pull Request?** Una propuesta para integrar los cambios de una rama en otra (normalmente `main`), que permite revisarlos antes.
12. **¿Qué hace `git rm --cached`?** Deja de seguir un fichero en Git sin borrarlo del disco.
13. **¿Qué comprueba `check_entorno.py --strict` y por qué falla fuera del venv?** Comprueba la versión de Python y que el intérprete es el de un `.venv`. Con `--strict` un entorno incorrecto es un error (no solo un aviso), así que fuera del venv falla.
14. **¿Qué cambia en `sys.executable` al activar el venv?** Pasa de apuntar al Python global a apuntar a `.venv/Scripts/python.exe` (o `.venv/bin/python`).
15. **¿Por qué se sube `.env.example` pero no `.env`?** `.env.example` solo contiene los nombres de las variables (documenta qué hace falta); `.env` contiene los valores reales, que son secretos.